

Combinatorial Objects

Permutations

k-element Subsets

Gray Codes

Generating, Ranking, Unranking

Numbers 1, 2, ..., N

can be perceived as just labels or indexes of some other items/objects $x_1, x_2, \dots, x_N$.

All ideas discussed here apply to the permutations of $\{1, 2, \dots, N\}$ and to the permutations of $\{x_1, x_2, \dots, x_N\}$ in the same way.

Example:

Permutations of set
of size 3

set {1, 2, 3}

{1, 2, 3}

{1, 3, 2}

{2, 1, 3}

{2, 3, 1}

{3, 1, 2}

{3, 2, 1}

index { 1, 2, 3 }

set { Ann, Bob, Don }

{Ann, Bob, Don}

{Ann, Don, Bob}

{Bob, Ann, Don}

{Bob, Don, Ann}

{Don, Ann, Bob}

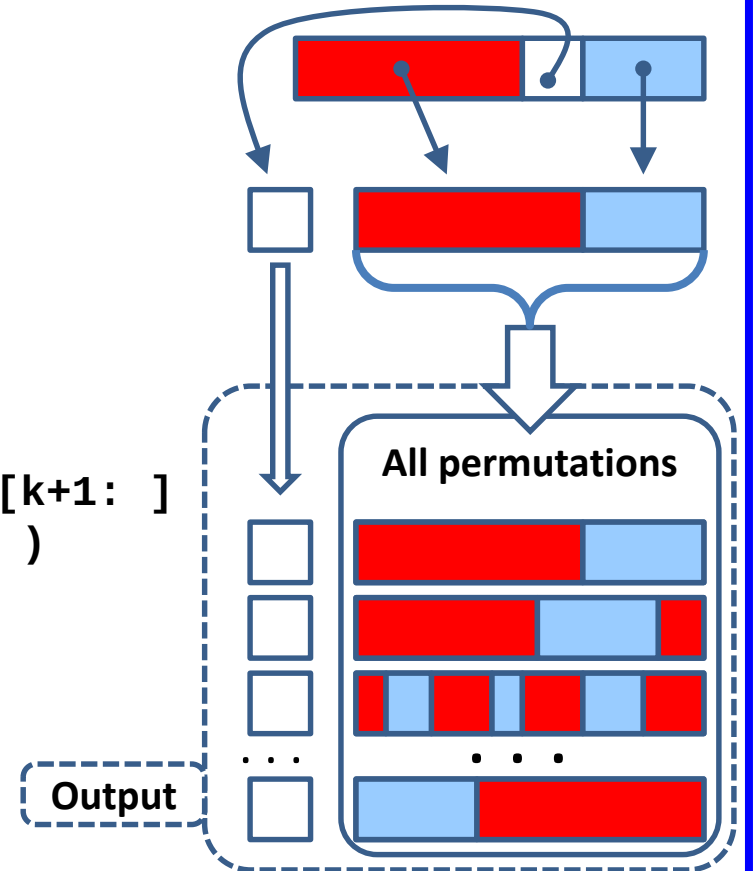
{Don, Bob, Ann}

Studying permutations of {1,2,3,4,5} -- list of permutations with their ranks

0 (1 2 3 4 5)	24 (2 1 3 4 5)	48 (3 1 2 4 5)	72 (4 1 2 3 5)	96 (5 1 2 3 4)
1 (1 2 3 5 4)	25 (2 1 3 5 4)	49 (3 1 2 5 4)	73 (4 1 2 5 3)	97 (5 1 2 4 3)
2 (1 2 4 3 5)	26 (2 1 4 3 5)	50 (3 1 4 2 5)	74 (4 1 3 2 5)	98 (5 1 3 2 4)
3 (1 2 4 5 3)	27 (2 1 4 5 3)	51 (3 1 4 5 2)	75 (4 1 3 5 2)	99 (5 1 3 4 2)
4 (1 2 5 3 4)	28 (2 1 5 3 4)	52 (3 1 5 2 4)	76 (4 1 5 2 3)	100 (5 1 4 2 3)
5 (1 2 5 4 3)	29 (2 1 5 4 3)	53 (3 1 5 4 2)	77 (4 1 5 3 2)	101 (5 1 4 3 2)
6 (1 3 2 4 5)	30 (2 3 1 4 5)	54 (3 2 1 4 5)	78 (4 2 1 3 5)	102 (5 2 1 3 4)
7 (1 3 2 5 4)	31 (2 3 1 5 4)	55 (3 2 1 5 4)	79 (4 2 1 5 3)	103 (5 2 1 4 3)
8 (1 3 4 2 5)	32 (2 3 4 1 5)	56 (3 2 4 1 5)	80 (4 2 3 1 5)	104 (5 2 3 1 4)
9 (1 3 4 5 2)	33 (2 3 4 5 1)	57 (3 2 4 5 1)	81 (4 2 3 5 1)	105 (5 2 3 4 1)
10 (1 3 5 2 4)	34 (2 3 5 1 4)	58 (3 2 5 1 4)	82 (4 2 5 1 3)	106 (5 2 4 1 3)
11 (1 3 5 4 2)	35 (2 3 5 4 1)	59 (3 2 5 4 1)	83 (4 2 5 3 1)	107 (5 2 4 3 1)
12 (1 4 2 3 5)	36 (2 4 1 3 5)	60 (3 4 1 2 5)	84 (4 3 1 2 5)	108 (5 3 1 2 4)
13 (1 4 2 5 3)	37 (2 4 1 5 3)	61 (3 4 1 5 2)	85 (4 3 1 5 2)	109 (5 3 1 4 2)
14 (1 4 3 2 5)	38 (2 4 3 1 5)	62 (3 4 2 1 5)	86 (4 3 2 1 5)	110 (5 3 2 1 4)
15 (1 4 3 5 2)	39 (2 4 3 5 1)	63 (3 4 2 5 1)	87 (4 3 2 5 1)	111 (5 3 2 4 1)
16 (1 4 5 2 3)	40 (2 4 5 1 3)	64 (3 4 5 1 2)	88 (4 3 5 1 2)	112 (5 3 4 1 2)
17 (1 4 5 3 2)	41 (2 4 5 3 1)	65 (3 4 5 2 1)	89 (4 3 5 2 1)	113 (5 3 4 2 1)
18 (1 5 2 3 4)	42 (2 5 1 3 4)	66 (3 5 1 2 4)	90 (4 5 1 2 3)	114 (5 4 1 2 3)
19 (1 5 2 4 3)	43 (2 5 1 4 3)	67 (3 5 1 4 2)	91 (4 5 1 3 2)	115 (5 4 1 3 2)
20 (1 5 3 2 4)	44 (2 5 3 1 4)	68 (3 5 2 1 4)	92 (4 5 2 1 3)	116 (5 4 2 1 3)
21 (1 5 3 4 2)	45 (2 5 3 4 1)	69 (3 5 2 4 1)	93 (4 5 2 3 1)	117 (5 4 2 3 1)
22 (1 5 4 2 3)	46 (2 5 4 1 3)	70 (3 5 4 1 2)	94 (4 5 3 1 2)	118 (5 4 3 1 2)
23 (1 5 4 3 2)	47 (2 5 4 3 1)	71 (3 5 4 2 1)	95 (4 5 3 2 1)	119 (5 4 3 2 1)

*# For each item in myList, run recursion on myList
with that item removed.
Prepend the removed item to each permutation returned
from the recursion.*

```
def allperm ( myList ):  
  
    if len(myList) == 1:  
        return [myList]  
  
    result = []  
    for k in range( len(myList) ):  
        item = myList[k]  
        shorterList = myList[ :k] + myList[k+1: ]  
        shorterPerms = allperm( shorterList )  
        for perm in shorterPerms:  
            result.append( [item] + perm )  
  
    return result
```



Poor time and space complexity, because of multiple lists generation.

Permutations of
{1,2,3,4,5,6,7,8,9}

Permutation:
(5 1 8 3 9 7 6 4 2)
Next permutation:
(5 1 8 4 2 3 6 7 9)



Lexicographical order of permutations

Next permutation of
(5 1 8 3 9 7 6 4 2) :

1.
Identify last increasing neighbour pair -- 3 and 9
(5 1 8 3 9 7 6 4 2)



2.
Swap 3 with the smallest value bigger than 3
to the right of 3:
(5 1 8 4 9 7 6 3 2)

3. Reverse the sequence to the right of 4
(= to the right of the original position of 3)
(5 1 8 4 2 3 6 7 9)



```

def nextperm( perm ):
    n = len(perm)
    # start at the last position
    j = n-1
    # find last ascending pair
    while True:
        if j == 0:
            return False # no next permutation
        if perm[j-1] > perm[j]:
            j -= 1
        else: break
    j1 = j-1 # remember position
    # find smallest bigger element than perm[j1] to the right of j1
    while j < n and perm[j] > perm[j1]: j += 1
    # index of that element:
    j -= 1
    perm[j], perm[j1] = perm[j1], perm[j] # swap
    # reverse sequence to the right of j1
    j1 += 1; j = n-1
    while j1 < j:
        perm[j], perm[j1] = perm[j1], perm[j]
        j1 += 1; j -= 1
    return True

```

The rank of permutation $(3,2,5,4,1)$ in the list of all permutations of $\{1,2,3,4,5\}$ is 59.

General strategy:

1. Note that the list of all permutations is divided into a number of blocks.
2. Establish the pattern by which the permutations are divided into blocks.
3. Note that this pattern has recursive character.
4. Using the established pattern, count (recursively) the number of blocks which precede the given permutation $(3,2,5,4,1)$ in the list of all permutations . This number is equal to the rank of the subset.

24	(2 1	3 4 5)
25	(2 1	3 5 4)
26	(2 1	4 3 5)
27	(2 1	4 5 3)
28	(2 1	5 3 4)
29	(2 1	5 4 3)

Note the regular sizing of the blocks.

```

def rankPermutation( perm ):
    n = len( perm )
    if n == 1: return 0

    rank = (perm[0]-1) * factorial(n-1)

    # consider the permutation without the 1st element
    perm1 = perm[1:] # copy w/o perm[0]
    # "normalize" the resulting permutation
    # for recursive processing
    for j in range(len(perm1)):
        if perm1[j] > perm[0]:
            perm1[j] -= 1

    return rank + rankPermutation( perm1 )

```

$\text{rank}((5 \ 1 \ 8 \ 3 \ 9 \ 7 \ 6 \ 4 \ 2)) = 4 \times 8! + \text{rank}((1 \ 7 \ 3 \ 8 \ 6 \ 5 \ 4 \ 2))$

← decreased by 1


```

def unrankPerm( rank, permLen ):
    n = permLen # just synonym
    if n == 1: return [1] # simplest possible permutation

    # count how many blocks of size n-1
    # would fit into list of all permutations
    # before the given rank
    blocksCount = rank // factorial(n-1) # integer div

    # construct the first element of the permutation
    firstElem = blocksCount + 1

    # calculate remaining rank to feed into recursion
    rank = rank % factorial(n-1)

    # exploit recursion
    perm = unrankPerm( rank, n-1)

    # "fit" the returned permutation to current size n
    for j in range( len(perm) ):
        if perm[j] >= firstElem:
            perm[j] += 1

    return [firstElem] + perm # concatenate lists

```

Studying subsets of $\{1,2,\dots,9\}$

Set: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Example subsets:

1, 2, 3, 4, 5, 6, 7, 8, 9

$A = \{1\}$	1 0 0 0 0 0 0 0 0 _{Bin}	= 128 _{Dec}
$A = \{1, 2\}$	1 1 0 0 0 0 0 0 0 _{Bin}	= 192 _{Dec}
$A = \{2, 4, 7, 9\}$	0 1 0 1 0 0 1 0 1 _{Bin}	= 165 _{Dec}
$A = \{6\}$	0 0 0 0 0 1 0 0 0 _{Bin}	= 8 _{Dec}
$A = \{6, 7\}$	0 0 0 0 0 1 1 0 0 _{Bin}	= 12 _{Dec}
$A = \{6, 7, 8\}$	0 0 0 0 0 1 1 1 0 _{Bin}	= 14 _{Dec}
$A = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$	1 1 1 1 1 1 1 1 1 _{Bin}	= 511 _{Dec}

Studying k -subsets of $\{1, 2, \dots, N\}$

Numbers $1, 2, \dots, N$

can be perceived as just labels or indexes of some other items/objects $x_1, x_2, \dots, x_N$.

All ideas discussed here apply to the subsets of $\{1, 2, \dots, N\}$
and to the subsets of $\{x_1, x_2, \dots, x_N\}$, in the same way.

Example:

set $\{1, 2, 3, 4, 5\}$

index $\{1, 2, 3, 4, 5\}$
set $\{\text{Ann, Bob, Don, Ema, Jan}\}$

2-subsets of
5-element sets

$\{1, 2\}$
 $\{1, 3\}$
 $\{1, 4\}$
 $\{1, 5\}$

 $\{2, 3\}$
 $\{2, 4\}$
 $\{2, 5\}$

 $\{3, 4\}$
 $\{3, 5\}$

 $\{4, 5\}$

$\{\text{Ann, Bob}\}$
 $\{\text{Ann, Don}\}$
 $\{\text{Ann, Ema}\}$
 $\{\text{Ann, Jan}\}$

 $\{\text{Bob, Don}\}$
 $\{\text{Bob, Ema}\}$
 $\{\text{Bob, Jan}\}$

 $\{\text{Don, Ema}\}$
 $\{\text{Don, Jan}\}$

 $\{\text{Ema, Jan}\}$

List all k-subsets of $\{1, 2, \dots, N\}$

All 3-subsets of $\{1, 2, \dots, 6\}$

$\{1, 2, 3\}$
 $\{1, 2, 4\}$
 $\{1, 2, 5\}$
 $\{1, 2, 6\}$

$\{1, 3, 4\}$ $\{2, 3, 4\}$
 $\{1, 3, 5\}$ $\{2, 3, 5\}$
 $\{1, 3, 6\}$ $\{2, 3, 6\}$

$\{1, 4, 5\}$ $\{2, 4, 5\}$ $\{3, 4, 5\}$
 $\{1, 4, 6\}$ $\{2, 4, 6\}$ $\{3, 4, 6\}$

$\{1, 5, 6\}$ $\{2, 5, 6\}$ $\{3, 5, 6\}$ $\{4, 5, 6\}$

$\{1, 2, 3\}$
 $\{1, 2, 4\}$
 $\{1, 2, 5\}$
 $\{1, 2, 6\}$
 $\{1, 2, 7\}$

$\{1, 3, 4\}$
 $\{1, 3, 5\}$
 $\{1, 3, 6\}$
 $\{1, 3, 7\}$

$\{1, 4, 5\}$
 $\{1, 4, 6\}$
 $\{1, 4, 7\}$

$\{1, 5, 6\}$
 $\{1, 5, 7\}$

$\{1, 6, 7\}$

All 3-subsets of $\{1, 2, \dots, 7\}$

$\{2, 3, 4\}$
 $\{2, 3, 5\}$
 $\{2, 3, 6\}$
 $\{2, 3, 7\}$

$\{2, 4, 5\}$
 $\{2, 4, 6\}$
 $\{2, 4, 7\}$

$\{2, 5, 6\}$
 $\{2, 5, 7\}$

$\{2, 6, 7\}$

$\{3, 4, 5\}$
 $\{3, 4, 6\}$
 $\{3, 4, 7\}$

$\{3, 5, 6\}$
 $\{3, 5, 7\}$

$\{3, 6, 7\}$

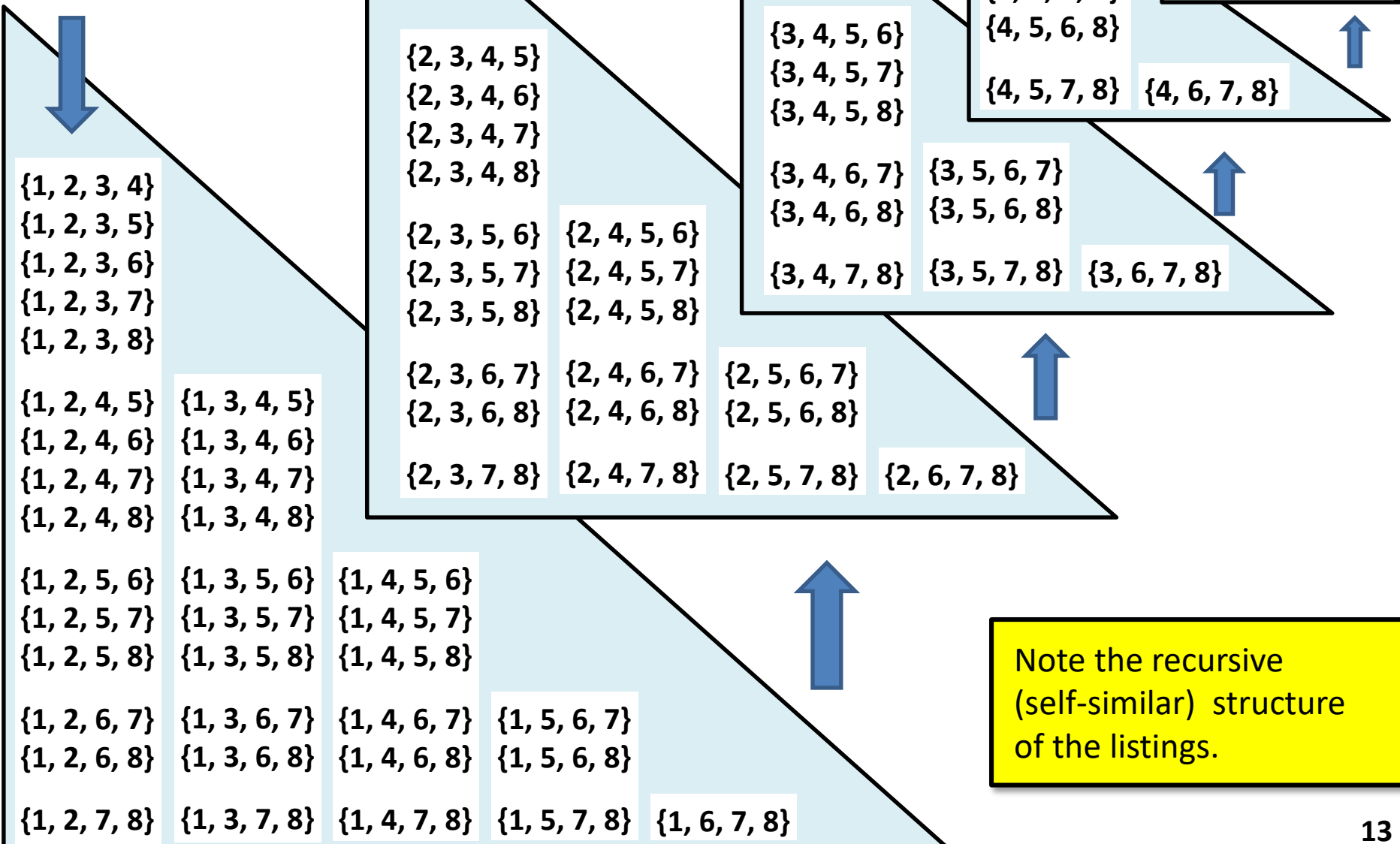
$\{4, 5, 6\}$
 $\{4, 5, 7\}$

$\{4, 6, 7\}$

$\{5, 6, 7\}$

Note the recursive (self-similar) structure of the listings.

List all 4-subsets of $\{1, 2, \dots, 8\}$



Note the recursive (self-similar) structure of the listings.

List all k -subsets of $\{1, 2, \dots, N\}$

Idea:

*# For each item I in the set generate all subsets
of size $k-1$ using only the elements to the right of I
(with higher index in the list)
and prepend I to each of the generated subsets.*

```
def k_subsets2( set, k ):
```

```
    # manage obvious edge cases
```

```
    if k == 0:
```

```
        return [[]]
```

```
    if k == len(set):
```

```
        return [set]
```

```
    # compose the result
```

```
    result = []
```

```
    for i in range( len(set) ):
```

```
        smallerSubsets = k_subsets2( set[i+1:], k-1 )
```

```
        for subset in smallerSubsets:
```

```
            result.append( [set[i]] + subset )
```

```
    return result
```

Poor time and space complexity, because of multiple lists generation.

List all k -subsets of $\{1, 2, \dots, N\}$

```
# Collect the items of the subset in a single result list.
# Process lists from the end to the beginning,
# with decreasing remaining depth to simplify the code.
# No additional index calculations! Cool, isn't it? :-)

def k_subsets3b( set, i_end, result, remainingDepth ):
    if remainingDepth < 0:      # note the zero!
        print( result )      # or add to some global variable
        return

    for i in range( i_end, remainingDepth-1, -1 ): # go backwards
        result[remainingDepth] = set[i]
        k_subsets3b( set, i-1, result, remainingDepth-1 )

# call:
set = [ 1,2,3, ... ]
k = ...
k_subsets3b( set, len(set)-1, [0]*k, k-1 )
```

See attached allksubs.py for more code variants.

Appropriate time and space complexity.

List all k -subsets of $\{1, 2, \dots, N\}$ using characteristic vector

Example: All 3-subsets of $\{1, 2, 3, 4, 5, 6\}$

characteristic vector subset rank

1 1 1 0 0 0 [1, 2, 3] 0

1 1 0 1 0 0 [1, 2, 4] 1

1 1 0 0 1 0 [1, 2, 5] 2

1 1 0 0 0 1 [1, 2, 6] 3

1 0 1 1 0 0 [1, 3, 4] 4

1 0 1 0 1 0 [1, 3, 5] 5

1 0 1 0 0 1 [1, 3, 6] 6

1 0 0 1 1 0 [1, 4, 5] 7

1 0 0 1 0 1 [1, 4, 6] 8

1 0 0 0 1 1 [1, 5, 6] 9

0 1 1 1 0 0 [2, 3, 4] 10

0 1 1 0 1 0 [2, 3, 5] 11

0 1 1 0 0 1 [2, 3, 6] 12

0 1 0 1 1 0 [2, 4, 5] 13

0 1 0 1 0 1 [2, 4, 6] 14

0 1 0 0 1 1 [2, 5, 6] 15

0 0 1 1 1 0 [3, 4, 5] 16

0 0 1 1 0 1 [3, 4, 6] 17

0 0 1 0 1 1 [3, 5, 6] 18

0 0 0 1 1 1 [4, 5, 6] 19

Identify a k -subset by its **characteristic (indicator)** vector.
That is, a 0/1 vector of length N containing exactly k 1s.

Generate all characteristic vectors, in descending lexicographical order, to represent all k -subsets, from $\{1, 2, \dots, k\}$ to $\{N-k+1, N-k+2, \dots, N\}$, that is from $(1, 1, \dots, 1, 0, \dots, 0, 0)$ to $(0, 0, \dots, 0, 1, \dots, 1, 1)$.

Abstain from recursion, iteration also works in this case.

List all k -subsets of $\{1, 2, \dots, N\}$ using characteristic vector

```
def nextChi( chi, k, N ):
    # skip all 1s at the end:
    j1 = N-1
    while chi[j1] == 1: j1 -= 1

    # no more subsets?
    if j1 == N-k-1: return False

    # next subset
    j0 = j1-1
    while chi[j0] == 0: j0 -= 1
    chi[j0], chi[j0+1] = 0, 1 # move 1 to the right

    # move remaining 1s from the end to just behind current 1
    numOfEndOnes = N-j1-1
    for j in range( j0+2, j0+2 + numOfEndOnes ): chi[j] = 1
    for j in range( j0+2 + numOfEndOnes, N ): chi[j] = 0
    return True
```

See attached allksubs.py for more code variants.

Appropriate time and space complexity.

List all k -subsets of $\{1, 2, \dots, N\}$ using characteristic vector

```
def k_subsets4( set, k ):
    N = len(set)
    if k == N:
        print( *set ); return

    chi = [1]*k + [0]*(N-k)
    rank = 0
    while True:
        print( *chi, end = ' ' )
        print( [set[k] for k in range(N) if chi[k] == 1 ], rank )
        if nextChi( chi, k, N ) == False: break
        rank += 1
```

See attached allksubs.py for more code variants.

Appropriate time and space complexity.

Studying 4-subsets of $\{1,2,\dots,12\}$

All 4-subsets of $\{1,2,\dots,12\}$ are (on this and on the next 8 slides):

0 {1 2 3 4}	17 {1 2 5 6}	30 {1 2 7 8}	45 {1 3 4 5}	60 {1 3 6 7}
1 {1 2 3 5}	18 {1 2 5 7}	31 {1 2 7 9}	46 {1 3 4 6}	61 {1 3 6 8}
2 {1 2 3 6}	19 {1 2 5 8}	32 {1 2 7 10}	47 {1 3 4 7}	62 {1 3 6 9}
3 {1 2 3 7}	20 {1 2 5 9}	33 {1 2 7 11}	48 {1 3 4 8}	63 {1 3 6 10}
4 {1 2 3 8}	21 {1 2 5 10}	34 {1 2 7 12}	49 {1 3 4 9}	64 {1 3 6 11}
5 {1 2 3 9}	22 {1 2 5 11}		50 {1 3 4 10}	65 {1 3 6 12}
6 {1 2 3 10}	23 {1 2 5 12}	35 {1 2 8 9}	51 {1 3 4 11}	
7 {1 2 3 11}		36 {1 2 8 10}	52 {1 3 4 12}	66 {1 3 7 8}
8 {1 2 3 12}	24 {1 2 6 7}	37 {1 2 8 11}		67 {1 3 7 9}
	25 {1 2 6 8}	38 {1 2 8 12}	53 {1 3 5 6}	68 {1 3 7 10}
9 {1 2 4 5}	26 {1 2 6 9}		54 {1 3 5 7}	69 {1 3 7 11}
10 {1 2 4 6}	27 {1 2 6 10}	39 {1 2 9 10}	55 {1 3 5 8}	70 {1 3 7 12}
11 {1 2 4 7}	28 {1 2 6 11}	40 {1 2 9 11}	56 {1 3 5 9}	
12 {1 2 4 8}	29 {1 2 6 12}	41 {1 2 9 12}	57 {1 3 5 10}	71 {1 3 8 9}
13 {1 2 4 9}			58 {1 3 5 11}	72 {1 3 8 10}
14 {1 2 4 10}		42 {1 2 10 11}	59 {1 3 5 12}	73 {1 3 8 11}
15 {1 2 4 11}		43 {1 2 10 12}		74 {1 3 8 12}
16 {1 2 4 12}				
		44 {1 2 11 12}		

Studying 4-subsets of $\{1,2,\dots,12\}$

The vertical spaces remind about the regularity patterns in the list.

75 {1 3 9 10}	88 {1 4 6 7}	103 {1 4 9 10}	120 {1 5 8 9}
76 {1 3 9 11}	89 {1 4 6 8}	104 {1 4 9 11}	121 {1 5 8 10}
77 {1 3 9 12}	90 {1 4 6 9}	105 {1 4 9 12}	122 {1 5 8 11}
	91 {1 4 6 10}	106 {1 4 10 11}	123 {1 5 8 12}
78 {1 3 10 11}	92 {1 4 6 11}	107 {1 4 10 12}	
79 {1 3 10 12}	93 {1 4 6 12}	108 {1 4 11 12}	124 {1 5 9 10}
			125 {1 5 9 11}
80 {1 3 11 12}	94 {1 4 7 8}	109 {1 5 6 7}	126 {1 5 9 12}
	95 {1 4 7 9}	110 {1 5 6 8}	
81 {1 4 5 6}	96 {1 4 7 10}	111 {1 5 6 9}	127 {1 5 10 11}
82 {1 4 5 7}	97 {1 4 7 11}	112 {1 5 6 10}	128 {1 5 10 12}
83 {1 4 5 8}	98 {1 4 7 12}	113 {1 5 6 11}	
84 {1 4 5 9}		114 {1 5 6 12}	129 {1 5 11 12}
85 {1 4 5 10}	99 {1 4 8 9}		
86 {1 4 5 11}	100 {1 4 8 10}	115 {1 5 7 8}	
87 {1 4 5 12}	101 {1 4 8 11}	116 {1 5 7 9}	
	102 {1 4 8 12}	117 {1 5 7 10}	
		118 {1 5 7 11}	
		119 {1 5 7 12}	

Studying 4-subsets of $\{1,2,\dots,12\}$

130 {1 6 7 8}	145 {1 7 8 9}	158 {1 8 10 11}	165 {2 3 4 5}
131 {1 6 7 9}	146 {1 7 8 10}	159 {1 8 10 12}	166 {2 3 4 6}
132 {1 6 7 10}	147 {1 7 8 11}		167 {2 3 4 7}
133 {1 6 7 11}	148 {1 7 8 12}	160 {1 8 11 12}	168 {2 3 4 8}
134 {1 6 7 12}			169 {2 3 4 9}
	149 {1 7 9 10}	161 {1 9 10 11}	170 {2 3 4 10}
135 {1 6 8 9}	150 {1 7 9 11}	162 {1 9 10 12}	171 {2 3 4 11}
136 {1 6 8 10}	151 {1 7 9 12}		172 {2 3 4 12}
137 {1 6 8 11}		163 {1 9 11 12}	
138 {1 6 8 12}	152 {1 7 10 11}		173 {2 3 5 6}
	153 {1 7 10 12}	164 {1 10 11 12}	174 {2 3 5 7}
139 {1 6 9 10}			175 {2 3 5 8}
140 {1 6 9 11}	154 {1 7 11 12}		176 {2 3 5 9}
141 {1 6 9 12}			177 {2 3 5 10}
	155 {1 8 9 10}		178 {2 3 5 11}
142 {1 6 10 11}	156 {1 8 9 11}		179 {2 3 5 12}
143 {1 6 10 12}	157 {1 8 9 12}		
144 {1 6 11 12}			

Studying 4-subsets of $\{1,2,\dots,12\}$

180 {2 3 6 7}	195 {2 3 9 10}	208 {2 4 6 7}	223 {2 4 9 10}
181 {2 3 6 8}	196 {2 3 9 11}	209 {2 4 6 8}	224 {2 4 9 11}
182 {2 3 6 9}	197 {2 3 9 12}	210 {2 4 6 9}	225 {2 4 9 12}
183 {2 3 6 10}		211 {2 4 6 10}	
184 {2 3 6 11}	198 {2 3 10 11}	212 {2 4 6 11}	226 {2 4 10 11}
185 {2 3 6 12}	199 {2 3 10 12}	213 {2 4 6 12}	227 {2 4 10 12}
186 {2 3 7 8}	200 {2 3 11 12}	214 {2 4 7 8}	228 {2 4 11 12}
187 {2 3 7 9}		215 {2 4 7 9}	
188 {2 3 7 10}	201 {2 4 5 6}	216 {2 4 7 10}	229 {2 5 6 7}
189 {2 3 7 11}	202 {2 4 5 7}	217 {2 4 7 11}	230 {2 5 6 8}
190 {2 3 7 12}	203 {2 4 5 8}	218 {2 4 7 12}	231 {2 5 6 9}
	204 {2 4 5 9}		232 {2 5 6 10}
191 {2 3 8 9}	205 {2 4 5 10}	219 {2 4 8 9}	233 {2 5 6 11}
192 {2 3 8 10}	206 {2 4 5 11}	220 {2 4 8 10}	234 {2 5 6 12}
193 {2 3 8 11}	207 {2 4 5 12}	221 {2 4 8 11}	
194 {2 3 8 12}		222 {2 4 8 12}	

Studying 4-subsets of $\{1,2,\dots,12\}$

235 {2 5 7 8}	250 {2 6 7 8}	265 {2 7 8 9}	278 {2 8 10 11}
236 {2 5 7 9}	251 {2 6 7 9}	266 {2 7 8 10}	279 {2 8 10 12}
237 {2 5 7 10}	252 {2 6 7 10}	267 {2 7 8 11}	
238 {2 5 7 11}	253 {2 6 7 11}	268 {2 7 8 12}	280 {2 8 11 12}
239 {2 5 7 12}	254 {2 6 7 12}		
		269 {2 7 9 10}	281 {2 9 10 11}
240 {2 5 8 9}	255 {2 6 8 9}	270 {2 7 9 11}	282 {2 9 10 12}
241 {2 5 8 10}	256 {2 6 8 10}	271 {2 7 9 12}	
242 {2 5 8 11}	257 {2 6 8 11}		283 {2 9 11 12}
243 {2 5 8 12}	258 {2 6 8 12}	272 {2 7 10 11}	
		273 {2 7 10 12}	284 {2 10 11 12}
244 {2 5 9 10}	259 {2 6 9 10}		
245 {2 5 9 11}	260 {2 6 9 11}	274 {2 7 11 12}	285 {3 4 5 6}
246 {2 5 9 12}	261 {2 6 9 12}		286 {3 4 5 7}
		275 {2 8 9 10}	287 {3 4 5 8}
247 {2 5 10 11}	262 {2 6 10 11}	276 {2 8 9 11}	288 {3 4 5 9}
248 {2 5 10 12}	263 {2 6 10 12}	277 {2 8 9 12}	289 {3 4 5 10}
			290 {3 4 5 11}
249 {2 5 11 12}	264 {2 6 11 12}		291 {3 4 5 12}

Studying 4-subsets of $\{1,2,\dots,12\}$

292 {3 4 6 7}	307 {3 4 9 10}	319 {3 5 7 8}	334 {3 6 7 8}
293 {3 4 6 8}	308 {3 4 9 11}	320 {3 5 7 9}	335 {3 6 7 9}
294 {3 4 6 9}	309 {3 4 9 12}	321 {3 5 7 10}	336 {3 6 7 10}
295 {3 4 6 10}		322 {3 5 7 11}	337 {3 6 7 11}
296 {3 4 6 11}	310 {3 4 10 11}	323 {3 5 7 12}	338 {3 6 7 12}
297 {3 4 6 12}	311 {3 4 10 12}		
		324 {3 5 8 9}	339 {3 6 8 9}
298 {3 4 7 8}	312 {3 4 11 12}	325 {3 5 8 10}	340 {3 6 8 10}
299 {3 4 7 9}		326 {3 5 8 11}	341 {3 6 8 11}
300 {3 4 7 10}	313 {3 5 6 7}	327 {3 5 8 12}	342 {3 6 8 12}
301 {3 4 7 11}	314 {3 5 6 8}		
302 {3 4 7 12}	315 {3 5 6 9}	328 {3 5 9 10}	343 {3 6 9 10}
	316 {3 5 6 10}	329 {3 5 9 11}	344 {3 6 9 11}
303 {3 4 8 9}	317 {3 5 6 11}	330 {3 5 9 12}	345 {3 6 9 12}
304 {3 4 8 10}	318 {3 5 6 12}		
305 {3 4 8 11}		331 {3 5 10 11}	346 {3 6 10 11}
306 {3 4 8 12}		332 {3 5 10 12}	347 {3 6 10 12}
		333 {3 5 11 12}	348 {3 6 11 12}

Studying 4-subsets of $\{1,2,\dots,12\}$

349 {3 7 8 9}	362 {3 8 10 11}	375 {4 5 7 8}	390 {4 6 7 8}
350 {3 7 8 10}	363 {3 8 10 12}	376 {4 5 7 9}	391 {4 6 7 9}
351 {3 7 8 11}		377 {4 5 7 10}	392 {4 6 7 10}
352 {3 7 8 12}	364 {3 8 11 12}	378 {4 5 7 11}	393 {4 6 7 11}
		379 {4 5 7 12}	394 {4 6 7 12}
353 {3 7 9 10}	365 {3 9 10 11}		
354 {3 7 9 11}	366 {3 9 10 12}	380 {4 5 8 9}	395 {4 6 8 9}
355 {3 7 9 12}		381 {4 5 8 10}	396 {4 6 8 10}
	367 {3 9 11 12}	382 {4 5 8 11}	397 {4 6 8 11}
356 {3 7 10 11}		383 {4 5 8 12}	398 {4 6 8 12}
357 {3 7 10 12}	368 {3 10 11 12}		
		384 {4 5 9 10}	399 {4 6 9 10}
358 {3 7 11 12}	369 {4 5 6 7}	385 {4 5 9 11}	400 {4 6 9 11}
	370 {4 5 6 8}	386 {4 5 9 12}	401 {4 6 9 12}
359 {3 8 9 10}	371 {4 5 6 9}		
360 {3 8 9 11}	372 {4 5 6 10}	387 {4 5 10 11}	402 {4 6 10 11}
	373 {4 5 6 11}	388 {4 5 10 12}	403 {4 6 10 12}
361 {3 8 9 12}	374 {4 5 6 12}		
		389 {4 5 11 12}	404 {4 6 11 12}

Studying 4-subsets of $\{1,2,\dots,12\}$

405 {4 7 8 9}	418 {4 8 10 11}	430 {5 6 8 9}	444 {5 7 9 10}
406 {4 7 8 10}	419 {4 8 10 12}	431 {5 6 8 10}	445 {5 7 9 11}
407 {4 7 8 11}		432 {5 6 8 11}	446 {5 7 9 12}
408 {4 7 8 12}	420 {4 8 11 12}	433 {5 6 8 12}	
			447 {5 7 10 11}
409 {4 7 9 10}	421 {4 9 10 11}	434 {5 6 9 10}	448 {5 7 10 12}
410 {4 7 9 11}	422 {4 9 10 12}	435 {5 6 9 11}	
411 {4 7 9 12}		436 {5 6 9 12}	449 {5 7 11 12}
	423 {4 9 11 12}		
412 {4 7 10 11}		437 {5 6 10 11}	450 {5 8 9 10}
413 {4 7 10 12}	424 {4 10 11 12}	438 {5 6 10 12}	451 {5 8 9 11}
			452 {5 8 9 12}
414 {4 7 11 12}	425 {5 6 7 8}	439 {5 6 11 12}	
	426 {5 6 7 9}		453 {5 8 10 11}
415 {4 8 9 10}	427 {5 6 7 10}	440 {5 7 8 9}	454 {5 8 10 12}
416 {4 8 9 11}	428 {5 6 7 11}	441 {5 7 8 10}	
417 {4 8 9 12}	429 {5 6 7 12}	442 {5 7 8 11}	455 {5 8 11 12}
		443 {5 7 8 12}	

Studying 4-subsets of $\{1,2,\dots,12\}$

456 {5 9 10 11}	469 {6 7 11 12}	480 {7 8 9 10}	490 {8 9 10 11}
457 {5 9 10 12}		481 {7 8 9 11}	491 {8 9 10 12}
	470 {6 8 9 10}	482 {7 8 9 12}	
458 {5 9 11 12}	<u>471 {6 8 9 11}</u>		492 {8 9 11 12}
	472 {6 8 9 12}	483 {7 8 10 11}	
459 {5 10 11 12}		484 {7 8 10 12}	493 {8 10 11 12}
	473 {6 8 10 11}		
460 {6 7 8 9}	474 {6 8 10 12}	485 {7 8 11 12}	494 {9 10 11 12}
461 {6 7 8 10}			
462 {6 7 8 11}	475 {6 8 11 12}	486 {7 9 10 11}	
463 {6 7 8 12}		487 {7 9 10 12}	
	476 {6 9 10 11}		
464 {6 7 9 10}	477 {6 9 10 12}	488 {7 9 11 12}	
465 {6 7 9 11}			
466 {6 7 9 12}	478 {6 9 11 12}	489 {7 10 11 12}	
467 {6 7 10 11}	479 {6 10 11 12}		
468 {6 7 10 12}			

The rank of subset $\{6,8,9,11\}$ in all 4-subsets of $\{1,2,\dots,12\}$ is 471 (see previous slide).

General strategy:

1. Note that the list of all 4-subsets is divided into a number of blocks.
2. Establish the pattern by which the 4-subsets are divided into blocks.
3. Note that this pattern has recursive character.
4. Using the established pattern, count (recursively) the number of blocks (on all significant recursion levels) which precede the given subset $\{6,8,9,11\}$ in the list of all subsets. This number is equal to the rank of the subset.

The rank of subset $\{6,8,9,11\}$ in all 4-subsets of $\{1,2,\dots,12\}$ is 471 (see earlier slide).

The minimum item in $\{6,8,9,11\}$ is 6.

Therefore $\{6,8,9,11\}$ is preceded in the list by all 4- subsets which contain values

- 1 and bigger
- 2 and bigger
- 3 and bigger
- 4 and bigger
- 5 and bigger

Specifically, those are:

0 {1 2 3 4}	165 {2 3 4 5}	285 {3 4 5 6}	369 {4 5 6 7}	425 {5 6 7 8}
1 {1 2 3 5}	166 {2 3 4 6}	286 {3 4 5 7}	370 {4 5 6 8}	426 {5 6 7 9}
...	...	...	...	...
...	...	...	...	...
164 {1 10 11 12}	284 {2 10 11 12}	368 {3 10 11 12}	424 {4 10 11 12}	459 {5 10 11 12}

The size of each of these 5 blocks is computed on next two slides.

Spoiler: Each size is a binomial coefficient.

The rank of subset $\{6,8,9,11\}$ in all 4-subsets of $\{1,2,\dots,12\}$ is 471 (see earlier slide).

0 {1 2 3 4}
 1 {1 2 3 5}
 ...
 ...
 164 {1 10 11 12}

Block 1. All 4-subsets which contain 1 and bigger values.

Value 1 is present in all subsets in the block.

When we remove 1 from each item in the block, we find that the size of the block is equal to the number of all 3-subsets of the set $\{2,3,4,\dots,12\}$.

Formally, that number is the same as the number of all 3-subsets of the set $\{1,2,3,\dots,11\}$ *).

And that, in turn, is equal to $\text{binCoeff}(11,3) = 11!/(3!*8!) = 165$

165 {2 3 4 5}
 166 {2 3 4 6}
 ...
 ...
 284 {2 10 11 12}

Block 2. All 4-subsets which contain 2 and bigger values.

Value 2 is present in all subsets in the block.

When we remove 2 from each item in the block, we find that the size of the block is equal to the number of all 3-subsets of the set $\{3,4,5,\dots,12\}$.

Formally, that number is the same as the number of all 3-subsets of the set $\{1,2,3,\dots,10\}$.

And that, in turn, is equal to $\text{binCoeff}(10,3) = 10!/(3!*7!) = 120$

*) Should be obvious, as the size of sets $\{2,3,4,\dots,12\}$ and $\{1,2,3,\dots,11\}$ is clearly the same.

The rank of subset $\{6,8,9,11\}$ in all 4-subsets of $\{1,2,\dots,12\}$ is 471 (see earlier slide).

285 {3 4 5 6}
 286 {3 4 5 7}
 ...
 ...
 368 {3 10 11 12}

Block 3. All 4-subsets which contain 3 and bigger values.

The size of the block is equal to the number of all 3-subsets of the set $\{4,5,6,\dots,12\}$.

Formally, that number is the same as the number of all 3-subsets of the set $\{1,2,3,\dots,9\}$.

And that, in turn, is equal to $\text{binCoeff}(9,3) = 9!/(3!*6!) = 84$

369 {4 5 6 7}
 370 {4 5 6 8}
 ...
 ...
 424 {4 10 11 12}

Block 4. All 4-subsets which contain 4 and bigger values.

Formally, the number of those subsets is the same as the number of all 3-subsets of the set $\{1,2,3,\dots,8\}$.

And that is equal to $\text{binCoeff}(8,3) = 8!/(3!*5!) = 56$

425 {5 6 7 8}
 426 {5 6 7 9}
 ...
 ...
 459 {5 10 11 12}

Block 5. All 4-subsets which contain 5 and bigger values.

Formally, the number of those subsets is the same as the number of all 3-subsets of the set $\{1,2,3,\dots,7\}$.

And that is equal to $\text{binCoeff}(7,3) = 7!/(3!*4!) = 35$

The rank of subset $\{6,8,9,11\}$ in all 4-subsets of $\{1,2,\dots,12\}$ is 471.

The subset $\{6,8,9,11\}$ is preceded by 5 blocks which total size is
 $165 + 120 + 84 + 56 + 35 = 460$.

Thus, the rank of $\{6,8,9,11\}$ is 460 or bigger.

The 4-subset $\{6,8,9,11\}$ is itself in the block 6 which contains values 6 and higher:

460 $\{6\ 7\ 8\ 9\}$

461 $\{6\ 7\ 8\ 10\}$

...

...

479 $\{6\ 10\ 11\ 12\}$

The rank of $\{6,8,9,11\}$ in all 4-subsets of $\{1,2,\dots,12\}$ is equal to
 $460 +$ the rank of $\{6,8,9,11\}$ in all 4-subsets of $\{6,7,8,\dots,12\}$.

Note that the value 6 is common in all subsets in this block.
 Remove it from the subsets and from the set $\{6,7,8,\dots,12\}$.

Now, recursion kicks in.

Therefore:

A. The rank of $\{6,8,9,11\}$ in all 4-subsets of $\{6,7,8,\dots,12\}$ is equal to
 the rank of $\{8,9,11\}$ in all 3-subsets of $\{7,8,\dots,12\}$.

B. The rank of $\{8,9,11\}$ in all 3-subsets of $\{7,8,\dots,12\}$ is equal to
 the rank of $\{2,3,5\}$ in all 3-subsets of $\{1,2,\dots,6\}$.

(just formally subtract 6 from all elements in the subset and the set $\{7,8,\dots,12\}$)

The rank of subset $\{6,8,9,11\}$ in all 4-subsets of $\{1,2,\dots,12\}$ is 471.

The subset $\{6,8,9,11\}$ is preceded by 5 blocks which total size is
 $165 + 120 + 84 + 56 + 35 = 460$.

Thus, the rank of $\{6,8,9,11\}$ is 460 or bigger.

The 4-subset $\{6,8,9,11\}$ is itself in the block 6 which contains values 6 and higher:

460 {6 7 8 9}

461 {6 7 8 10}

...

...

479 {6 10 11 12}

Previous slide A+B:

The rank of $\{6,8,9,11\}$ in all 4-subsets of $\{6,7,8,\dots,12\}$ is equal to
the rank of $\{2,3,5\}$ in all 3-subsets of $\{1,2,\dots,6\}$.

Apply recursion -- same problem structure, smaller parameters.

The rank of $\{2,3,5\}$ in all 3-subsets of $\{1,2,\dots,6\}$ is 11.

Studying 3-subsets of $\{1,2,\dots,6\}$

Level 1

All 3-subsets of $\{1,2,\dots,6\}$ are

0 {1 2 3}	10 {2 3 4}	16 {3 4 5}	19 {4 5 6}
1 {1 2 4}	<u>11 {2 3 5}</u>	17 {3 4 6}	
2 {1 2 5}	12 {2 3 6}		
3 {1 2 6}		18 {3 5 6}	
	13 {2 4 5}		
4 {1 3 4}	14 {2 4 6}		
5 {1 3 5}			
6 {1 3 6}	15 {2 5 6}		
7 {1 4 5}			
8 {1 4 6}			
9 {1 5 6}			

The rank of subset $\{2,3,5\}$ in all 3-subsets of $\{1,2,\dots,6\}$ is 11 (see previous slide).

The minimum item in $\{2,3,5\}$ is 1.

Therefore $\{2,3,5\}$ is preceded in the list by all 3-subsets which contain values
-- 1 and bigger

Specifically, that is:

0 {1 2 3}

1 {1 2 4}

...

9 {1 5 6}

Block 1. All 3-subsets which contain 1 and bigger values.

Value 1 is present in all subsets in the block.

When we remove 1 from each item in the block, we find that the size of the block is equal to the number of all 2-subsets of the set $\{2,3,4,\dots,6\}$.

Formally, that number is the same as the number of all 2-subsets of the set $\{1,2,3,\dots,5\}$.

And that, in turn, is equal to $\text{binCoeff}(5,2) = 5!/(2!*3!) = 10$

The rank of subset $\{2,3,5\}$ in all 3-subsets of $\{1,2,\dots,6\}$ is 11.

The subset $\{2,3,5\}$ is preceded by 1 blocks which size is 10.

Thus, the rank of $\{2,3,5\}$ is 10 or bigger.

The 3-subset $\{2,3,5\}$ is itself in the block 2 which contains values 2 and higher:

10 $\{2\ 3\ 4\}$

11 $\{2\ 3\ 5\}$

...

...

19 $\{4\ 5\ 6\}$

The rank of $\{2,3,5\}$ in all 3-subsets of $\{1,2,\dots,6\}$ is equal to 10 + the rank of $\{2,3,5\}$ in all 3-subsets of $\{2,3,4,\dots,6\}$.

Note that the value 2 is common in all subsets in this block. Remove it from the subsets and from the set $\{2,3,4,\dots,6\}$.

Therefore:

A. The rank of $\{2,3,5\}$ in all 3-subsets of $\{2,3,4,\dots,6\}$ is equal to the rank of $\{3,5\}$ in all 2-subsets of $\{3,4,\dots,6\}$.

B. The rank of $\{3,5\}$ in all 2-subsets of $\{3,4,\dots,6\}$ is equal to the rank of $\{1\ 3\}$ in all 2-subsets of $\{1,2,\dots,4\}$.

(Just formally subtract 2 from all elements in the subset and the set $\{3,4,\dots,6\}$.)

Studying 2-subsets of $\{1,2,\dots,4\}$

Level 2

All 2-subsets of $\{1,2,\dots,4\}$ are

0 $\{1\ 2\}$

1 $\{1\ 3\}$

2 $\{1\ 4\}$

3 $\{2\ 3\}$

4 $\{2\ 4\}$

5 $\{3\ 4\}$

The rank of subset $\{1,3\}$ in all 2-subsets of $\{1,2,\dots,4\}$ is 1.

0 $\{1\ 2\}$

1 $\{1\ 3\}$

...

2 $\{1\ 4\}$

The minimum item in $\{1, 3\}$ is 1.

Therefore $\{1,3\}$ is in the list, in the first block.

In other words, it is preceded by 0 blocks which contain value 0 and higher.

(Value 0 cannot appear in the subset).

The rank of $\{1,3\}$ in the first block in the list of all 2-subsets of $\{1,2,\dots,4\}$ is equal to the rank of $\{3\}$ in the list of all 1-subsets of $\{2,\dots,4\}$.

That is the same as the rank of $\{2\}$ in the list of all 1-subsets of $\{1,\dots,3\}$.
(Just subtract 1 from all elements in the subset and the set $\{1,\dots,3\}$.)

All 1-subsets of $\{1,2,\dots,3\}$ are

0 $\{1\}$

1 $\{2\}$

2 $\{3\}$

The rank of subset $\{2\}$ in all 1-subsets of $\{1,2,\dots,3\}$ is 1.

Finding the rank of 1-element subset $\{a\}$ of the set $\{1,2,\dots,X\}$ is easy,
just return $a-1$.

To conclude:

Finding the rank of a subset required computing recursively
the size of blocks which preceded the given subset in the
lexicographically ordered list of all subsets of the given size.

The computations on consecutive levels of recursion yielded total sizes
 $460 + 10 + 0 + 1 = 471$.

```

def rankSubset( subset, n ):
    k = len(subset)
    if k == 1: return subset[0] - 1

    rank = 0
    # total number of all subsets containing
    # values 1, 2, ..., subset[0]-1, which precede the given subset
    # in the list of all subsets lexicographically sorted
    for i in range(1, subset[0]):
        rank += binCoeff( n-i, k-1 )

    # exclude first elem from the subset array
    # and "normalize" the input for recursion
    subset1 = subset[1:] # copy of subset[1..k]
    for j in range( len(subset1) ):
        subset1[j] -= subset[0]
    n1 = n - (subset[0])

    # and recurse
    return rank + rankSubset( subset1, n1)

```

```

def unrankSubset ( rank, n, k ):
    if k == 1: return [rank+1]    # list with single value

    # jump over appropriate number of blocks
    # which precede the subset with the given rank
    # and simultaneously construct value subset[0]
    n1 = n-1          # next n value in recursive call
    subset0 = 1
    while True:
        blockSize = binCoeff( n1, k-1 )
        if blockSize <= rank:
            rank -= blockSize
            subset0 += 1
            n1 -= 1
        else: break

    subsetRec = unrankSubset ( rank, n1, k-1 )
    for j in range( len(subsetRec) ):
        subsetRec[j] += subset0

    return [subset0] + subsetRec  # list concatenation

```


Gray code examples

```
def grayCode( n ):
    if n == 1: return ["0", "1"]
    gc0 = grayCode(n-1)
    gc1 = list(reversed(gc0))
    for i in range(len(gc0)):
        gc0[i] = "0 "+gc0[i]
    for i in range(len(gc1)):
        gc1[i] = "1 "+gc1[i]

    return gc0+gc1

for i in range (1,6):
    for z in grayCode( i ):
        print(z)
    print()
```

